High-Level Programming Review

Jesferrel Porman Sijabat

Chapters:

- Strings
- Functions
- Classes
- Pointers

Strings

C-string

```
#include <cstring>
```

C++ string

```
#include <string>
```

C-Strings

With a \0 (Null) at the end.

So, always **add +1** of the actual length of the string if we want to specify the length.

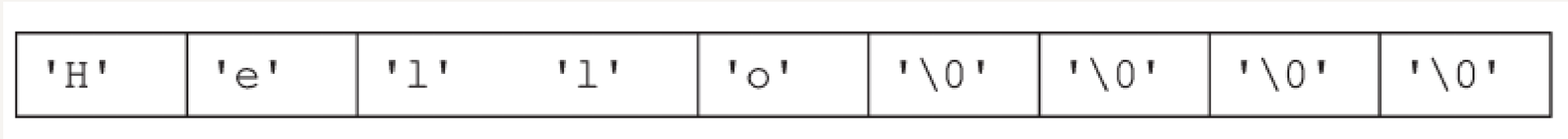
Array of Characters:

```
char greetings[6] = {'H', 'E', 'L', 'L', 'O', '\0'};
```

```
char greetings[] = "Hello";
```

String literal: Double quotation
We don't need to specify the length, compiler automatically adds a null.

```
char greetings[9] = "Hello";
```



Empty slots are replaced with Nulls

```
4   #include <iostream>
5   using namespace std ;
6
7   main()
8   {
9     const int MAX_CHARACTERS = 10 ;
10    char first_name[ MAX_CHARACTERS + 1 ] ;
11
12    cout << "Enter your first name (maximum "
13         << MAX_CHARACTERS << " characters) " ;
14    cin >> first_name ;
15    cout << "Hello " << first_name << endl ;
16  }
```

→ Define array with length [11]

→ Stores user input into array

cin → Only reads until **whitespace**

Imagine if we want to take a *sentence* from the user...

.getline(*array, length, delimiter*)

```
4  #include <iostream>
5  using namespace std ;
6
7  main()
8  {
9    const int MAX_CHARACTERS = 10 ;
10   char first_name[ MAX_CHARACTERS + 1 ] ;
11
12   cout << "Enter your first name(maximum "
13        << MAX_CHARACTERS << " characters) " ;
14   cin.getline( first_name, MAX_CHARACTERS + 1, '\n' ) ;
15   cout << "Hello " << first_name << endl ;
16 }
```

Same number as the dimension of the array.

cin.getline() → Reads until **length is reached**, or **delimiter is pressed**

delimiter is OPTIONAL (if left blank, it will make Enter '\n' the delimiter)

WATCHOUT

```
6  main()
7  {
8    const int MAX_CHARACTERS = 20 ;
9    char student_name[ MAX_CHARACTERS + 1 ] ;
10   int student_number ;
11
12   cout << "Enter student number: " ;
13   cin >> student_number ;
14   cout << "Enter student first name and surname (maximum "
15        << MAX_CHARACTERS << " characters) " ;
16   cin.getline( student_name, MAX_CHARACTERS + 1 ) ;
17   cout << endl << "Data Entered:" << endl
18        << "Student Number: " << student_number << endl
19        << "Student Name: " << student_name << endl ;
20 }
```

Solution:

cin.ignore(80, '\n')

→ ignores up to 80 characters, until and including \n

Why not just .getline() on the first cin..?

Output:

```
Enter student number: 12345
Enter student first name and surname (maximum 20 characters)
Data Entered:
Student Number: 12345
Student Name:
```

1. You type **1, 2, 3, 4, 5.**
2. You press **Enter**. This adds the **\n** character immediately after the 5.
3. The input stream now contains: **12345\n**
4. The command **cin >> student_number** pulls only the numeric part (12345) out the stream to store it.
5. Because that command **stops reading** as soon as it hits a non-numeric character, the **\n is left behind,** sitting by itself at the very front of the stream.

C-String Function

```
// Don’t forget:
#include <cstring>
```

- strlen(str)
- strcpy(destination, source)
- strcat(str1, str2)
- strcmp(str1, str2)

strlen(str)

```
char name1[]   = "Sharon" ;
char name2[10] = "Mark" ;
int len ;
len = strlen( name1 ) ;
cout << setw( 3 ) << strlen( name1 )
     << setw( 3 ) << strlen( name2 )
     << setw( 3 ) << strlen( "Rob" )
     << setw( 3 ) << len ;
```

```
Output:
6
4
3
6
```

String Length → Returns **Number of Characters** in a string *excluding \0*

strcpy(dest, source)

```
char name1[] = "Sharon" ;
char name2[10] = "Mark" ;
// Copy the contents of name1 to name2.
strcpy( name2, name1 ) ;
// Restore the original name.
strcpy( name2, "Mark" ) ;
```

String Copy → Copies the contents of the **source** string to **destination** string

strcat(str1, str2)

```
char str1[15] = "first & " ;
char str2[] = "second" ;
strcat( str1, str2 ) ; // str1 is now "first & second".
```

String Concatenate → **Concatenates** 2 strings (adds them up) and stores them into str1.

Enough memory must be allocated to str1 to hold the result of the concatenation.

strcmp(str1, str2)

```
3  #include <iostream>
4  #include <cstring>
5  using namespace std ;
6
7  main()
8  {
9    char password[7] = "secret" ;
10   char user_input[81] ;
11   cout << "Enter Password: " ;
12   cin >>  user_input ;
13   if ( strcmp( password, user_input ) == 0 )
14     cout << "Correct password. Welcome to the system ..." << endl ;
15   else
16     cout << "Invalid password" << endl ;
17 }
```

This **Function** Returns:

- a **negative value**, if str1 < str2
- **0**, if str1 == str2
- a **positive value**, if str1 > str2

→ Usually used to compare if 2 strings have the same exact contents

Compared by its **ASCII value**.
ex: *"apple"* < *"banana"* because '**a**' (ASCII 97) is **less than** '**b**' (ASCII 98).

C-string Functions with n

`strncat(str1, str2, n)`

→ Appends the **first n characters** of str2 to str1.

`strncmp(str1, str2, n)`

→ Compares the **first n characters**

`strncpy(str1, str2, n)`

→ Copies the **first n characters** of str2 to str1

Summary C-string Functions

Function	Remark
strlen(str)	Finding the length of a C-string str
strcpy(str1, str2)	copies the contents of a C-string str2 to str1
strcat(str1, str2)	concatenates a C-string str2 to the end of the C-string str1
strcmp(str1, str2)	compares two null-terminated C-strings str1 and str2.
strncat(str1, str2, n)	Appends the first n characters of the C-string str2 to str1.
strncmp(str1, str2, n)	Identical to strcmp, except that at most n characters are compared.
strncpy(str1, str2, n)	Copies the first n characters of str2 into str1.

Converting C-string to Numbers

“123”

'1'	'2'	'3'	'\0'
49	50	51	0
00110001	00110010	00110011	00000000

Storage form:

→ **ASCII** Representation

123

00000000	01111011

→ **Binary**

atoi(str) : C-string to an integer (+- 2.14 billion)
atol(str) : C-string to a long integer (+- 9.22 quintillion)
atof(str) : C-string to a double float.

Example Code:

```
#include <cstdlib>

char str[] = “123”;

int int_number;
double double_number;

int_number = atoi(str);
double_number = atof(str);
```

C++ Strings

```
// Don't forget
#include <string>

string str1 = "ABCDEFG";
string str2(11, '-');           // Define a string of 11 dashes
string str3 = "This is the first part,"
              " and this is the second part";
```

- **Automatically** handles **memory management** (don't need to define array size)
- Hidden **null** terminators.

→ **Safer** and **Easier** to work with.

C++ String Functions

str is the Variable Name!

- str.assign(**param**)
- str.swap(target)
- Concatenate Strings (**+**)
- str.append(**param**)
- str.length()
- str.substr(**param**)
- String Searching
- String Comparison
- str.replace(**param**)
- str.erase(**param**)
- str.insert(**param**)

str.assign(param)

`str.assign(amount, char)`

`str.assign(target, index, amount)`

→ **Overloading**: The function behaves **differently** depending on the **parameters**.

```
24   // String assignment examples.
25   str1 = "ABCD" ;
26   str2.assign( 3, '.' ) ; // Assign 3 dots to str2.
27   cout << "After the 1st and 2nd assignments:" << endl
28        << "  str1=" << str1 << endl
29        << "  str2=" << str2 << endl ;
33   str5.assign( str1, 1, 3 ) ;  // Assign "BCD" to str5.
34   cout << "After the 3rd assignment:" << endl
35        << "  str5=" << str5 << endl ;
```

Output:

```
After the 1st and 2nd assignments:
  str1=ABCD
  str2=...
After the 3rd assignment:
  str5=BCD
```

→ a different way of assigning strings

str.swap(target)

```
41    str1.swap( str2 ) ; // swap str1 and str2.
42    cout << "After swapping str1 and str2:"  << endl
43         << "  str1=" << str1 << endl
44         << "  str2=" << str2 << endl ;
45 }
```

Output:

```
Before swapping str1 and str2:
  str1=ABCD
  str2=...
After swapping str1 and str2:
  str1=...
  str2=ABCD
```

→ Swaps the contents of the 2 strings

String Concatenation

```
string str1 = “ABCD”, str2, str3;
str2.assign(3, ‘.’);

str3 = str1 + str2;     // 1st concatenation
str3 += “etc.”;         // 2nd concatenation
```

Output:

```
After the 1st concatenation:
  str1=ABCD
  str2=...
  str3=ABCD...
After the 2nd concatenation:
  str3=ABCD...etc.
```

→ to add 2 strings together, we can just use **+**

+= Just like how we would do it with numbers.

str.append(param)

```
str.append(string)
str.append(target, index, amount)
str.append(amount, char)
```

→ Overloading

```
str3.append(“ ,etc.,etc. “);        //3rd concatenation

str4 = “It is near the end of the program”;
str3 = “This is “;
str3.append(str4, 11, 7);           //4th concatenation
str3.append(3, ‘.’);                //5th concatenation
```

Output:

```
After the 3rd concatenation:
  str3=ABCD...etc., etc., etc.
```

```
After the 4th concatenation:
  str3=This is the end
```

```
After the 5th concatenation:
  str3=This is the end...
```

str.length()

```
string str1 = “ABCDEFGH”;

// Store the length of str1
int len1 = str1.length();
```

str.at()

Changing the **First** and **Last** Index

```
str1[0] = ‘*’;
str1[len1-1] = ‘*’;
str1.at(0) = ‘A’;
str1.at(len1-1) = ‘H’;
```

There’s a chance of using [index] to fall out of bounds
→ **hard to track the bug**

Use **str.at()** because it’s **safer.**

str.substr(index, amount)

```
string str2 = "ABCDEFGH" ;
cout << "Demonstration of substr:" << endl << "  " ;
cout << "The first four characters of " << str2<< " are "
     <<  str2.substr( 0, 4 ) << endl << "  "
     << "The middle two characters of " << str2 << " are "
     <<  str2.substr( 3, 2 ) << endl << "  "
     << "The last three characters of " << str2 << " are "
     <<  str2.substr( 5,3 ) << endl ;
```

Output:

```
Demonstration of substr:
  The first four characters of ABCDEFGH are ABCD
  The middle two characters of ABCDEFGH are DE
  The last three characters of ABCDEFGH are FGH
```

str.replace(param)

target.replace(**index, amount, source, index, amount**)
target.replace(**index, amount, source**)

Source → **Variable** or a **String**

```
string str1 =  " ABCDE "  ;
string str2 =  " abcdefghij "  ;

str1.replace( 1, 3, str2, 2, 4 ) ;
cout <<  " After the 1st replacement: "  << endl
     <<  "  str1= "  << str1 << endl ;

str1 =  " ABCDE "  ;
str1.replace( 1, 3, str2 ) ;
cout <<  " After the 2nd replacement: "  << endl
     <<  "  str1= "  << str1 << endl ;
```

Output:

```
After the 1st replacement:
  str1=AcdefE
```

```
After the 2nd replacement:
  str1=AabcdefghijE
```

str.erase(param)

```
target.erase(index)
target.erase(index, amount)
target.erase()
```

Erases from **Index** to **End**
Erases from **Index** to the **Specified Amount**
Erases the **Whole Thing**

```
str1.erase( 9 ) ;
cout << " After the 1st erase: " << endl
     << "  str1= " << str1 << endl ;
// str1 = Aabcdefgh
str1.erase( 4, 2 ) ;
cout << " After the 2nd erase: " << endl
     << "  str1= " << str1 << endl ;

str1.erase() ;
cout << " After the 3rd erase: " << endl
     << "  str1= " << str1 << endl ;
if ( str1.empty() )
    cout << " str1 is empty " << endl ;
else
    cout << " str1 is not empty " << endl ;
```

Output:

```
After the 1st erase:
  str1=Aabcdefgh
After the 2nd erase:
  str1=Aabcfgh
After the 3rd erase:
  str1=
str1 is empty
```

String Searching

```
string str1 = “ABCDEFABCDEF”;
int p;

                                    Output:
p = str1.find(“CDE”);               2
p = str1.rfind(“CDE”);              8
p = str1.find_first_of(“ED”);       3
p = str1.find_last_of(“ED”);        10
p = str1.find_first_not_of(“ABC”)   3
p = str1.find_last_not_of(“ABC”)    11
```

String Searching

str1.**find**(“CDE”) finds the **first** occurence of the string CDE
str1.**rfind**(“CDE”) finds the **last** occurence of the string CDE
str1.**find_first_of**(“ED”) find the **very first** “E” or “D” anywhere in the string
str1.**find_last_of**(“ED”) find the **very last** “E” or “D” anywhere in the string
str1.**find_first_not_of**(“ABC”) find the **first** occurence of **not** either “A”, “B”, or “C”
str1.**find_last_not_of**(“ABC”) find the **last** occurence of **not** either “A”, “B”, or “C”.

String Comparison

```
string str1 = “ABCDEFGH”;
string str2 = “BCD”;
int result = str1.compare(str2);
int resultSpecific = str1.compare(1, 3, str2)

// There are 2 ways, and they both mean the same thing
if (str1 == str2 || result == 0){
  cout << “str1 and str2 is equal” << endl;
} else if (str1 < str2 || result < 0){
  cout << “str1 is less than str2” << endl;
} else if (str1 > str2 || result > 0){
  cout << “str1 is more than str2“ << endl;
}
```

→ if **exactly** the same

- **resultSpecific** will have a **value of 0** because they are **EXACTLY** the same.
- **result** will be **< 0** because **A < B** in comparison.

ASCII

Alphabetical Order:
→ The **higher** the order, the **greater** the value (c > b > a)

Case-Sensitive:
→ All lowercase is **greater** than UPPERCASE (a > A)

Numbers:
→ Numbers are **less** than Letters

“100” < “2” evaluates true in a string condition.

OTHER C-STRING FUNCTIONS

Returns **True** or **False**:

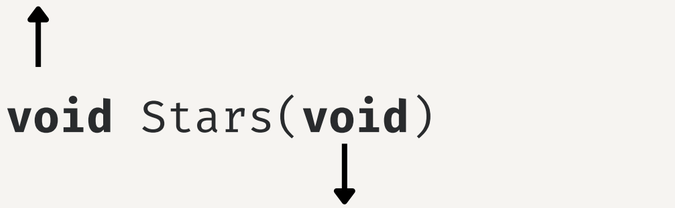
Function	Character set
`isalnum`	Alphanumeric character: A-Z, a-z, 0-9
`isalpha`	Alphabetic character: A-Z, a-z
`isascii`	ASCII character: ASCII codes 0-127
`iscntrl`	Control character: ASCII codes 0-31 or 127
`isdigit`	Decimal digit: 0-9
`isgraph`	Any printable character other than a space
`islower`	Lowercase letter: a-z
`isprint`	Any printable character, including a space
`ispunct`	Any punctuation character
`isspace`	Whitespace character: \t,\v,\f,\r,\n or space ASCII codes 9-13 or 32
`isupper`	Uppercase letter: A-Z
`isxdigit`	Hexadecimal digit: 0-9 and A-F

Function	Purpose
`tolower`	Converts an uppercase character to lowercase.
`toupper`	Converts a lowercase character to uppercase.

Functions

Blocks of **statements** called by **name** to carry out a **specific task**
(REUSABILITY)

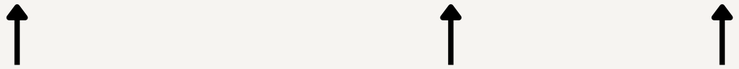
Returns **NOTHING**



```
void Stars(void)
```

Receive **NOTHING**

Returns **Integer** Receives **Integer**



```
int minimum_val(int num1, int num2){
  if (num1 > num2){
     return num2;
  } else {
     return num1;
  }
}

int main(){
  int val1 = 0;
  int val2 = 1;
  cout << “the minimum of “ << val1 << “and” << val2
       << “is” << minimum_val(val1,val2) << endl;
}
```

Default Values

```
void disp_chars(int num = 1, char ch) ;×
void disp_chars(int num, char ch = ' ') ;√
```

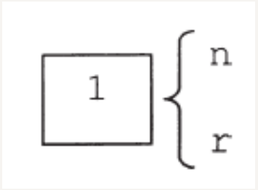
Function uses **Default Value** if **nothing** is passed when called

If you set a default value in the parameter,
then the parameters **TO THE RIGHT must** also
have a default value

Reference

```
int n = 1;
int& r = n;

n = 2;      // changes both n and r
```



- **r** is a **reference** to **n**
- **n** and **r** both refer to the **same value**
- **r** is not a copy of **n**, but is merely **another name** for **n**
- A **change** to **n** will also result in a **change** to **r**

Reference

```
6  void any_function ( int& p );
7
8  main()
9  {
10     int a = 1;
11     cout << "a is" << a << endl ;
12
13     any_function (a) ;
14
15     cout << "a is now " << a << endl  ;
16 }
17
18 void any_function( int& p )
19 {
20     cout << "p is" << p << endl ;
21     p = 2 ;
22 }
```

a and **p** refer to the **same Storage Location.**
Changing the value of **p** changes also the value of **a.**

Output:

```
a is 1
p is 1
a is now 2
```

Passing an Array

Can only be passed as **reference**. Prevents copying all array elements.

```
11  int main()
12  {
13      int values[10] = { 12, 4, 5, 3, 4, 0, 1, 8, 2, 3 } ;
14      int sum ;
15
16      sum = sum_array( values , 10 ) ;
17      cout << "The sum of the array elements is" << sum <<endl;
18  }
19
20  int sum_array ( const int array [], int no_of_elements)
21  {
22      int total = 0;
23
24      for( int index = 0 ; index < no_of_elements ; index++ )
25          total += array[index] ;
26      return total ;
27  }
```

WHAT DOES THIS MEAN?

- If we modify the array[], it will modify the ***values[10]***
- keyword **const**: array cannot be changed, thus protecting the ***values*** array.
- [] are necessary to indicate that the parameter **is a reference to an array.**

The *number of elements* is **not required** in the brackets for a *one-dimensional array* to handle different size array.

Passing a Multi-dimensional Array

```
int sum_array( const int array[][2], int no_of_rows)
{
    int total=0;

    for (int row =0; row < no_of_rows ;row++)
    {
        for( int col=0;col <2;col++)
            total += array[row][col];
    }
    return total;
}

main()
{
    int values[5][2] = { { 31, 14 },
                         { 51, 11 },
                         {  7, 10 },
                         { 13, 41 },
                         { 16, 18 } };
    int sum;

    sum= sum_array(values,5);
    cout<< "The sum of the array elements is " << sum << endl;
}
```

Output:

```
The sum of the array elements is 212
```

The Array parameter **MUST** contain the **DIMENSION SIZE** other than the first slot.

FYI

```
int values[5][2] = { { 31, 14 },
                     { 51, 11 },
                     {  7, 10 },
                     { 13, 41 },
                     { 16, 18 } };
```

8.3 Passing arguments

- The elements of a two-dimensional array are stored row by row in contiguous memory locations.

31	14	51	11	7	10	13	41	16	18

  - Any element *array[i][j]* in this situation has an offset i*2 + j from the starting memory location of *array[0][0].*

  - In order to calculate the offset, the number of columns (= 2) is required; hence the need for the compiler to know the value of the second dimension

`array[3][1]` (= 41) has an offset of 7 (= 3 * 2 + 1).

Mathematical Functions

#include <cmath>

```cpp
#include <cmath>
using namespace std ;

main()
{
    const double RADIANS_IN_A_DEGREE = 57.29578 ;

    double degrees, radians ;

    cout << "Input the angle in degrees:" ;
    cin >> degrees ;
    radians = degrees / RADIANS_IN_A_DEGREE ;
    cout << fixed << setprecision( 3 )
         << "sin(" << degrees << ")=" << sin(radians) << endl
         << "cos(" << degrees << ")=" << cos(radians) << endl
         << "tan(" << degrees << ")=" << tan(radians) << endl ;
}
```

Output:

```
Input the angle in degrees: 60
sin(60.000)= 0.866
cos(60.000)= 0.500
tan(60.000)= 1.732
```

Mathematical Functions

Generating random numbers: **srand(), rand()**

```
#include <cstdlib> // Required for rand() and srand()
#include <ctime>   // Required for time()

srand(time(0));    // time(0) gets the current system time in seconds

// rand() % 10 gives a number from 0 to 9. Adding 1 shifts it to 1 to 10.
int ranged_num = (rand() % 10) + 1;

// Generate a random number between 50 and 100
// Formula: (rand() % (max - min + 1)) + min
int custom_range_num = (rand() % (100 - 50 + 1)) + 50;
```

→ **srand()** generates **a seed** for the random number generator.
time(0) ensures that the seed is always **unique**.
→ **rand()** generates a random number.

Function Overloading

Same Function name, **different amount** of or **data types** of the parameters.
The compiler knows itself depending on the data passed on, which one to use.

Example:

```
str.append(string)
str.append(target, index, amount)
str.append(amount, char)

str.assign(amount, char)
str.assign(target, index, amount)
```

→ **Overloading**

→ **Overloading**

Function Overloading

```
6  int sum_array ( const: int array [] , int no_of_elements ) ;
7  // Purpose : Sums the elements of a 1-D integer array.
8  // Parameters: An array and the number of elements in the array.
9  // Returns : The sum of the array elements.
10 int sum_array( const int array[][2] int no_of_rows ) ;
11 // Purpose : Sums the elements of a 2-D integer array.
12 // Parameters: A 2-D array and the number of rows in the array.
13 // Returns : The sum of the array elements.
14
15 main()
16 {
17   int one_d_array[5] = { 0, 1, 2, 3, 4 } ;
18   int sum ;
19
20   sum = sum_array( one_d_array, 5 ) ;
21   cout << "The sum of the 1-D array elements is "
22        << sum << endl ;
23
24   int two_d_array[3][2] = { { 0, 1 },
25                             { 11. 12 },
26                             { 21, 22 } } ;
27
28   sum = sum_array( two_d_array, 3 ) ;
29   cout << "The sum of the 2-D array elements is " << sum << endl;
30 }
```

Output:

```
The sum of the 1-D array elements is 10
The sum of the 2-D array elements is 67
```

Recursion

A function is called inside its own function

$$n! \text{ is } \begin{cases} 1 \text{ when } n \text{ is } 0 \\ n * (n-1)! \text{ when } n>0 \end{cases}$$

```
unsigned int factorial(int n){
  if (n == 0){
    return 1;
  } else {
    return (n * factorial(n-1));
  }
}
```

unsigned means it can only hold non-negative values.

Block Scope

Variables defined in a block is accessible **only within** that block.
Block is statements enclosed in **curly braces { }**.

Global Scope

Variables defined **outside of the main()** from anywhere within the program

Block Scope

The scope of the variable x , *y*

```
void f ( int x )
{
    // x is accessible here.
    int y ;
    ...
    if( x == 1 )
    {
        int z;
        //x, y and z are accessible here.
        ...
    }// z is destroyed here.
    // x and y are accessible here, but z is not.
    ...
}// x and y are destroyed when the function terminates.
```

Global Scope

```
int g; // g is a global variable.

void f1() ;
void f2() ;

main
{
    int a ;
    // a and g are accessible here.
    ...
    // Program ends, a and g are destroyed.
}
void f1()
{
    int b ;
    // b and g are accessible here.
    ...
    // Function ends, b is destroyed.
}

void f1()
{
    // g is accessible here .
}
```

Reusing Variable Names

```
 6  int i = 1 ; // i is a global variable.
 7
 8  void f();
 9
10  main()
11  { // Start of program block.
12
13      cout << "Global variable i=" << i << endl;
14
15      int i = 2; // i is reused here.
16      cout << "Variable i declared in main() = "<< i <<end1;
17
18      // The global variable i can be accessed by using ::
19      cout << "Global variable i = " << ::i << end1 ;
```

Access the global variable by using ::

Classes

Class is **blueprint** that can be used to create many Instances of the Class *(REUSABILITY)*

Class Example

```
class Animal {
  // Private members can only be accessed from within the class itself
  private:
    string name;
    string species;
    unsigned int age; // Using unsigned since age cannot be negative
```

Private usually contains **member Data**.
→ Can **only be accessed** by **member Functions** of the class

Constructor

is a **Function** that **runs automatically** when an instance of the class is created.

Has the **same name** as the class, **no return** type.

```
public:
    Animal(string animalName, string animalSpecies, unsigned int animalAge) {
        name = animalName;
        species = animalSpecies;
        age = animalAge;
    }
```

Used to **Initialize** the values of the **Data Members**

→ by ***passing arguments*** through the class' ***parameters***.

Class Method

```
public:
    // Member Function (Method): Defines an action the object can perform
    void makeSound(string sound) {
        cout << name << " the " << species << " says: " << sound << "!" << endl;
    }

    void displayInfo() {
        cout << "Name: " << name << ", Species: " << species << ", Age: " << age
<< " years old." << endl;
    }
```

Public usually contains **member Functions** and the **Constructor**.
→ Can **accessed** from **outside** the class.

Through the **Member Functions**, we can modify the **Member Data**

Creating Objects

```
// We pass the starting values to the constructor
Animal cat("Luna", "Cat", 3);
Animal dog("Rex", "Dog", 5);

// Calling the member functions using the dot (.) operator
cat.displayInfo();
cat.makeSound("Meow");

dog.displayInfo();
dog.makeSound("Woof");
```

The **Animal class** is just the **template**
cat and **dog** are the actual *objects (instances)* created using that template.
→ They each hold their own unique set of name, species, and age data.

Overloading Constructors

```
// Used when no information is provided at creation.
    Animal() {
        name = "Unknown";
        species = "Unknown";
        age = 0;
    }
// Used when name and species are known, but we want to set a default age.
    Animal(string animalName, string animalSpecies) {
        name = animalName;
        species = animalSpecies;
        age = 1; // Defaulting to 1 year old
    }
```

Constructor is a **function** (can be **Overloaded**)

→ We can pick which contructor to use **depending** on the **parameters** passed on

Static Property

Only **one copy** of a static data member exists and is **shared by all objects** of a particular class.
→ **Shared Memory**

```
int Animal::totalAnimals = 0;                // initialize static property OUTSIDE
class Animal {
  private:
    string name;

    // This variable is shared across all Animal instances
    static int totalAnimals;

  public:
    Animal(string animalName) {.             // CONSTRUCTOR - always called
        name = animalName;
        totalAnimals++;                      // runs when new object is created
    }

    // A static method can only access static variables.
    // It can be called without needing a specific object.
    static int getTotalAnimals() {
        return totalAnimals;
    }
};
```

```
// Create a few animals
    Animal cat("Luna");
    Animal dog("Rex");
    Animal bird("Tweety");

// The shared counter has now been incremented 3 times
    cout << "Count after creating objects: " << Animal::getTotalAnimals() << endl;
```

Good to know

```
class Animal {
  private:
    int age;

  public:
    Animal(int startingAge) { age = startingAge; }
    void setAge(int newAge);
};

void Animal::setAge(int newAge) {
    age = newAge;
}
```

Writing the function **outside** of the class using the :: operator.
We can do **inline** void Animal::setAge(int newAge) to treat it as if the function is **in** the class body.
→ **Reduces** function call overhead.

Class Inheritance

allows a **new class** to be constructed using an **existing class** as a basis.

Base Class

```
class Animal {
// 'protected' means these variables are hidden from outside (like 'private'),
// BUT they are accessible to any derived child classes.
  protected:
    string name;
    int age;

  public:
    Animal(string animalName, int animalAge) {
    name = animalName;
    age = animalAge;
    }

    void eat() {
      cout << name << " is eating." << endl;
    }
};
```

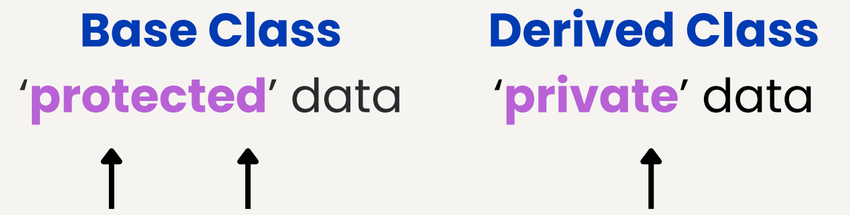
Derived Class

```
class Dog : public Animal {
  private:
   string breed;

  public:
// The derived class constructor MUST pass the data to the base class constructor.
// using an initialization list: : Animal(dogName, dogAge)
    Dog(string dogName, int dogAge, string dogBreed) : Animal(dogName, dogAge) {
     breed = dogBreed; // Initialize the Dog-specific property
    }

    void bark() {
// We can use 'name' here directly because it is 'protected' in the Animal class
      cout << name << " (a " << breed << ") says: Woof!" << endl;
    }
};
```

Create Objects




```
Dog myDog("Rex", 5, "Golden Retriever");
// Using inherited methods from the Animal base class
myDog.eat();

// Using specific methods from the derived classes
myDog.bark();
```

Override Methods

```
class Dog : public Animal {
  private:
   string breed;

  public:
    Dog(string dogName, int dogAge, string dogBreed) : Animal(dogName, dogAge) {
     breed = dogBreed;
    }

    void eat() {   // The same eat() function is being overriden in the Dog class
      cout << name << “is eating dog food” << endl;
    }
};
```

Types of Inheritance

Type of inheritance	Base class member access	Derived class member access
public	public	public
	protected	protected
	private	inaccessible
protected	public	protected
	protected	protected
	private	inaccessible
private	public	private
	protected	private
	private	inaccessible

class **Dog** : **public Animal**{}
class **Dog** : **protected Animal**{}
class **Dog** : **private Animal**{}

Pointers

& and *

Example

```
int var = 1;        // A standard integer variable.
int* ptr;           // ptr is a pointer to an int variable[cite: 88, 89].

ptr = &var;         // ptr now contains the memory address of var. (0012FF88)

cout << &var << endl; // e.g., 0012FF88
cout << ptr << endl;  // e.g., 0012FF88
cout << *ptr << endl;  // e.g., 1
```

Address Operator (&): Every variable is stored in a memory location with a **unique address**
int* ptr: ptr is a **pointer** to an int.
Deference Operator (*) : ***ptr** accesses the **actual value** stored at that memory address

With Constant

```
int var = 1;
int var2 = 2;
```

1. **Pointer to a constant integer**

```
const int* ptr1 = &var;
*ptr1 = 5;      ❌
ptr1 = &var2;   ✅
```

2. **Constant pointer to an integer**

```
int* const ptr1 = &var;
*ptr1 = 5;      ✅
ptr1 = &var2;   ❌
```

3. **Constant pointer to a constant integer**

```
const int* const ptr1 = &var;
*ptr1 = 5;      ❌
ptr1 = &var2;   ❌
```

Pointers with Array

```
int a[5] = {1, 2, 3, 4, 5};
```

The **name** of the array **a** itself is a **pointer** to the first index

```
  a    → 0012FF78
&a[0] → 0012FF78
```

So we can **dereference** with * to access the elements

```
*(a + 1) → a[1]
*(a + 2) → a[2]
```

but array pointers are constant

❌

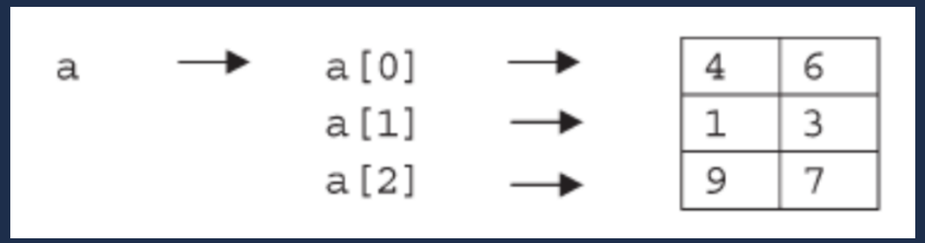
```
a++
a = a + 1
```

✅

```
int* p = a;    // initialize p as a pointer
p++;           // → a[1]
```

Multi-dimesional Array

```
int a[3][2] = {
        {4, 6},
        {1, 3},
        {9, 7}
    };
```




```
*a → {4,6}
*(a+1) → {1,3}
*(a+2) → {9,7}
```

Each row

```
**a    → a[0][0]
*a[1] → a[1][0]
*a[2] → a[2][0]
```

First Element of each row

```
*(a[0]+1) → a[0][1]
*(a[1]+1) → a[1][1]
*(a[2]+1) → a[2][1]
```

+1 Element of each row

Class and Struct Arrays

Access **data** and **functions** of Classes and Structures.

```
class Animal {
private:
    string species;
    int age;
public:
    Animal(string sp, int a) {
        species = sp;
        age = a;
    }
    void have_birthday() {
        age++;
    }
}
```

```
Animal my_pet("Golden Retriever", 3);

Animal* pet_ptr;
pet_ptr = &my_pet;

pet_ptr->have_birthday();
(*pet_ptr).have_birthday();
```

Pointer - Arguments

Using pointers as Functions Arguments

```
void swap_vals(float* ptr1, float* ptr2) {
    float temp = *ptr1;
    *ptr1 = *ptr2;
    *ptr2 = temp;
}

float num1 = 12.1;
float num2 = 6.4;

swap_vals(&num1, &num2);
```

Thank you for participating in **HSIYC AcademiaSphere's HLL Programming Review Class #2!**

Your feedback helps us evaluate the effectiveness of our review sessions and improve future academic support activities. This survey takes approximately 1-2 minutes to complete. We appreciate your honest responses!

